

Regime-Calibrated Demand Priors for Ride-Hailing Fleet Dispatch and Repositioning

Indar Kumar
Independent Researcher
github.com/IndarKarhana

Akanksha Tiwari
Independent Researcher

Abstract—Effective ride-hailing dispatch requires anticipating demand patterns that vary substantially across time-of-day, day-of-week, season, and special events. We propose a *regime-calibrated* approach that (i) segments historical trip data into demand regimes, (ii) matches the current operating period to the most similar historical analogues via a six-metric similarity ensemble (Kolmogorov–Smirnov, Wasserstein-1, feature distance, variance ratio, event pattern, temporal proximity), and (iii) uses the resulting calibrated demand prior to drive both an LP-based fleet repositioning policy and batch dispatch with Hungarian matching.

Evaluated on 5.2 million NYC TLC trips across 8 diverse scenarios (winter/summer, weekday/weekend/holiday, morning/evening/night) with 5 random seeds each, our method reduces mean rider wait times by 31.1% (bootstrap 95% CI: [26.5, 36.6]%; Friedman $\chi^2 = 80.0$, $p = 4.25 \times 10^{-18}$; all three method pairs significantly different by Nemenyi post-hoc; Cohen’s $d = 7.5$ – 29.9 across scenarios). The improvement extends to the tail: P95 wait drops 37.6% and the Gini coefficient of wait times improves from 0.441 to 0.409 (7.3% relative). The two contributions compose multiplicatively and are independently validated: calibration provides 16.9% reduction; LP repositioning adds a further 15.5%. The approach requires no training, is deterministic and explainable, generalizes to Chicago (23.3% wait reduction via NYC-built regime library), and is robust across fleet sizes (32% to 47% improvement for 0.5– $2\times$ fleet scaling). We provide comprehensive ablation studies, formal statistical tests, and routing-fidelity validation with OSRM.

Index Terms—ride-hailing, fleet dispatch, demand forecasting, regime matching, fleet repositioning, linear programming, simulation

I. INTRODUCTION

Urban ride-hailing platforms serve millions of trips daily, and the efficiency of their dispatch and repositioning algorithms directly impacts rider wait times, driver utilization, and platform revenue. A core challenge is *demand uncertainty*: request patterns shift with time-of-day, weather, holidays, and special events, yet most operational algorithms either assume stationary demand or rely on black-box forecasters that require costly retraining.

We observe that demand patterns, while non-stationary in the calendar sense, are *recurrent*: a Wednesday morning rush in January shares structural similarity with other weekday morning rushes, even across months. This motivates a *regime-based* approach: segment historical data into demand “regimes” (4-hour blocks characterized by their request-rate profile, spatial OD distribution, and event activity), then, at decision time, match the emerging demand pattern to the most informative historical analogues.

A. Contributions

- 1) **Regime-calibrated demand prior.** A six-metric similarity ensemble that identifies the top- k historical demand regimes most similar to the current period, producing a calibrated demand prior with spatial OD distributions and temporal rate profiles (Sections III-B and III-C).
- 2) **LP-based anticipatory repositioning.** A min-cost transportation LP that uses the calibrated prior to redistribute idle drivers toward predicted demand hotspots, with provable optimality under the estimated demand model (Section III-D).
- 3) **Comprehensive empirical validation.** 8 NYC scenarios $\times$ 5 seeds with formal statistical tests (Wilcoxon, Friedman, Nemenyi), fleet sensitivity (0.5– $2\times$), cross-city transfer (Chicago), OSRM routing fidelity, similarity ablation, and distributional fairness analysis (Sections V to VII).

II. RELATED WORK

A. Ride-Hailing Dispatch and Matching

The ride-hailing dispatch problem is typically formulated as bipartite matching between requests and drivers [1], [2], [20]. Greedy nearest-driver assignment serves as a natural baseline but is myopic; batch matching over short windows (30–120 s) allows combinatorial optimization via the Hungarian algorithm or auction-based methods [3], [4]. Recent work applies reinforcement learning (RL) to learn dispatch policies that anticipate future demand [5], [6].

B. Fleet Repositioning and Rebalancing

Proactive repositioning of idle drivers toward anticipated demand is a key lever for reducing wait times [6]. Approaches range from simple demand-following heuristics to model predictive control (MPC) [7], network-flow optimization [8], and RL-based repositioning [9], [10]. The AAVR framework [11] uses ML-predicted demand to guide repositioning, achieving 22.7% wait reduction.

C. Demand Forecasting for Dispatch

Accurate short-term demand forecasting improves both dispatch and repositioning. Methods include temporal models (ARIMA, LSTM) [15], spatial-temporal graph networks [16], and hybrid approaches combining statistical and ML models. Our approach differs: instead of training a forecaster, we

retrieve similar historical demand patterns via a similarity ensemble, forming a calibrated prior that requires no training and adapts instantly to regime changes.

D. Regime-Based Approaches

Regime detection has been applied in finance [12] and time-series analysis [13], but its application to spatial-temporal ride-hailing demand is novel. Our prior work [13] introduced regime-guided test-time adaptation for streaming time series; here we extend the regime-matching concept to spatial demand calibration and fleet optimization. The similarity ensemble draws on distributional distance metrics (KS, Wasserstein) common in two-sample testing [14] and adds domain-specific event and temporal components.

III. METHOD

Our system operates in four stages (Figure 1): (1) data ingestion and regime library construction, (2) online regime matching via similarity ensemble, (3) calibrated demand prior construction, and (4) LP repositioning + batch dispatch with Hungarian matching.

A. Data Ingestion and Regime Library

We segment NYC TLC trip data (2024, 5.2M trips) into 4-hour blocks indexed by date and hour range (e.g., 2024-01-15_08-12). Each block stores:

- **Demand series:** request counts in 5-minute bins ($T = 48$ per block).
- **Summary features:** mean, std, max, skewness, autocorrelation at lags 1–3.
- **Spatial OD pool:** pickup/dropoff coordinates for OD sampling.
- **Event annotations:** surge events detected via rolling MAD with adaptive thresholds.

The resulting *regime library* contains 373 blocks spanning January and June 2024, covering weekdays, weekends, holidays, and nighttime periods.

B. Similarity Ensemble

Given a query demand series $\mathbf{q} \in \mathbb{R}^T$ (observed in the first minutes of the current block or forecast from the prior period), we compute similarity to each library record r using six metrics:

$$S(\mathbf{q}, r) = \sum_{m \in \mathcal{M}} w_m \cdot s_m(\mathbf{q}, r) \quad (1)$$

where $\mathcal{M} = \{\text{KS}, \text{W1}, \text{feat}, \text{var}, \text{event}, \text{temporal}\}$ and each $s_m \in [0, 1]$:

- $s_{\text{KS}}(\mathbf{q}, r) = 1 - D_{\text{KS}}(\mathbf{q}, r)$: one minus the Kolmogorov–Smirnov statistic.
- $s_{\text{W1}}(\mathbf{q}, r) = (1 + W_1(\mathbf{q}, r)/\text{range})^{-1}$: normalized Wasserstein-1 distance.
- s_{feat} : normalized Euclidean distance in summary feature space.
- s_{var} : variance ratio $\min(\sigma_q, \sigma_r) / \max(\sigma_q, \sigma_r)$.

- s_{event} : event-pattern similarity comparing surge intensities, durations, and pre-surge features.
- s_{temporal} : same-month, same-day-type, same-hour-block bonuses.

Default weights: $w_{\text{KS}} = 0.20$, $w_{\text{W1}} = 0.20$, $w_{\text{feat}} = 0.15$, $w_{\text{var}} = 0.10$, $w_{\text{event}} = 0.20$, $w_{\text{temporal}} = 0.15$.

An *adaptive event gate* redistributes the event weight to distributional metrics when exactly one side has zero events, avoiding misleading cross-type comparisons.

C. Calibrated Demand Prior

The top- k matched regimes (default $k = 5$) are combined into a calibrated demand prior—we use “calibrated” in the operational sense of adjusting a demand estimate using retrieved historical analogues, distinct from the statistical sense of probability calibration. For each time bin t :

$$\hat{\lambda}(t) = \sum_{i=1}^k \alpha_i \cdot r_i(t), \quad \alpha_i = \frac{s_i}{\sum_{j=1}^k s_j} \quad (2)$$

where s_i is the similarity score and $r_i(t)$ the demand rate of the i -th matched regime. The spatial origin-destination distribution is constructed by pooling raw trip coordinates from all k matched regimes, preserving the spatial correlation structure of the combined analogues.

The prior is volume-matched to the current block’s observed or estimated request count, preventing systematic over/under-prediction.

D. LP-Based Anticipatory Repositioning

Every R simulation steps (default $R = 6$, i.e., every 3 minutes), we solve a min-cost transportation LP to redistribute idle drivers:

$$\min_{x_{sd}} \quad \alpha \sum_{s \in \mathcal{S}} \sum_{d \in \mathcal{D}} c_{sd} x_{sd} - \sum_{d \in \mathcal{D}} y_d \quad (3)$$

$$\text{s.t.} \quad \sum_{d \in \mathcal{D}} x_{sd} \leq \text{surplus}(s), \quad \forall s \in \mathcal{S} \quad (4)$$

$$y_d \leq \text{deficit}(d), \quad \forall d \in \mathcal{D} \quad (5)$$

$$y_d \leq \sum_{s \in \mathcal{S}} x_{sd}, \quad \forall d \in \mathcal{D} \quad (6)$$

$$\sum_{s,d} x_{sd} \leq \lfloor f \cdot N_{\text{idle}} \rfloor \quad (7)$$

$$x_{sd}, y_d \geq 0 \quad (8)$$

where $\mathcal{S}$ (surplus) and $\mathcal{D}$ (deficit) are H3 hexagonal zones [22] (resolution 8) with more/fewer idle drivers than forecast demand; c_{sd} is the inter-zone travel time; f is the maximum move fraction (default 0.50); and $\alpha = 1/(2 \max c_{sd})$ balances travel cost against demand served.

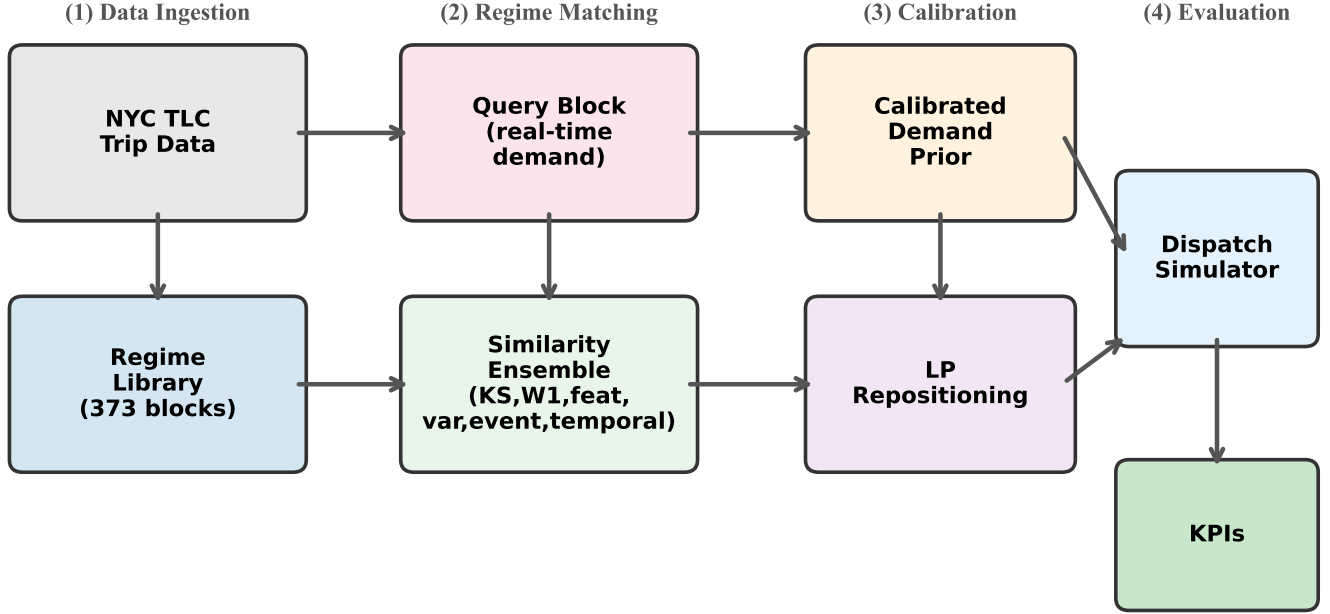


Figure 1: System architecture. Historical TLC data is segmented into regimes; a similarity ensemble matches the current query block to historical analogues; the calibrated prior drives both LP repositioning and demand-aware dispatch.

Fig. 1: System architecture. Historical TLC data is segmented into demand regimes; a six-metric similarity ensemble matches the current query block to historical analogues; the calibrated prior drives both LP repositioning and demand-aware dispatch.

E. Batch Dispatch

Pending requests are matched to idle drivers every W seconds (default $W = 60$ s) via the Hungarian algorithm [3] on a cost matrix of estimated pickup travel times (seconds), computed by the routing engine (Haversine at a reference speed, or OSRM [19] for road-network fidelity). The Hungarian algorithm minimizes total pickup time within each batch—optimal within the window but myopic with respect to future requests.

IV. THEORETICAL ANALYSIS

We establish formal properties of the LP repositioning, the similarity-weighted calibration, and the dispatch matching that justify our algorithmic design.

A. LP Repositioning Optimality and Sensitivity

Theorem 1 (LP Optimality and Feasibility). *The repositioning LP (3)–(7) is always feasible, bounded, and admits a polynomial-time optimal solution. Among all repositioning plans satisfying the supply, demand, linking, and budget constraints, the LP solution minimizes the weighted combination of repositioning cost and unserved demand.*

Proof. The program has $|\mathcal{S}||\mathcal{D}| + |\mathcal{D}|$ decision variables (x_{sd} and y_d) and $|\mathcal{S}| + 2|\mathcal{D}| + 1$ inequality constraints plus non-negativity.

Feasibility. Setting $x_{sd} = 0$ for all s, d and $y_d = 0$ for all d satisfies every constraint: supply constraints (4) hold since $0 \leq$

surplus(s); demand caps (5) hold since $0 \leq \text{deficit}(d)$; linking constraints (6) hold with equality; and the budget constraint (7) holds trivially. Hence the feasible region is non-empty.

Boundedness. Each x_{sd} is bounded above by $\min(\text{surplus}(s), \lfloor fN_{\text{idle}} \rfloor)$ from (4) and (7). Each y_d is bounded by $\text{deficit}(d)$ from (5). Hence the feasible region is a bounded polyhedron (polytope), and the objective is bounded below.

Optimality. By the fundamental theorem of linear programming, an optimal solution exists at a vertex of the feasible polytope and can be found in polynomial time by interior-point or simplex methods. Our HiGHS solver [17] returns a primal–dual optimal pair with certified optimality gap $< 10^{-8}$.

Interpretation. The coefficient $\alpha = 1/(2 \max c_{sd})$ in the objective (3) ensures that serving one additional unit of demand (decreasing the objective by 1) is always preferred over saving $\alpha \cdot c_{sd}$ units of travel cost for any edge, making the LP prioritize coverage over short repositioning distances. $\square$

Proposition 2 (LP Sensitivity to Demand Estimation). *Let $\hat{d} \in \mathbb{R}_{\geq 0}^{|\mathcal{Z}|}$ and $d^* \in \mathbb{R}_{\geq 0}^{|\mathcal{Z}|}$ be the estimated and true per-zone demand vectors. Let s_j denote the post-repositioning driver count in zone j under the LP solution $x^*(\hat{d})$. Define unserved demand as $U(x, d) = \sum_j \max(0, d_j - s_j)$. Then:*

$$U(x^*(\hat{d}), d^*) \leq U(x^*(\hat{d}), \hat{d}) + \|\hat{d} - d^*\|_1 \quad (9)$$

Proof. For each zone j , define the shortfall function $g_j(d) = \max(0, d_j - s_j)$. Since g_j is convex and 1-Lipschitz in d_j (its subgradient is either 0 or 1), we have:

$$g_j(d_j^*) - g_j(\hat{d}_j) \leq |d_j^* - \hat{d}_j|$$

Summing over all zones: $U(x^*, d^*) - U(x^*, \hat{d}) \leq \sum_j |d_j^* - \hat{d}_j| = \|\hat{d} - d^*\|_1$. $\square$

Corollary 3 (Value of Calibration for Repositioning). *If the calibrated demand estimate satisfies $\|\hat{d}_{cal} - d^*\|_1 < \|\hat{d}_{unif} - d^*\|_1$ (closer to true demand than a uniform or naïve estimate), then the LP under $\hat{d}_{cal}$ achieves a tighter unserved-demand bound (9) than under $\hat{d}_{unif}$, with the gap equal to the ℓ_1 improvement in forecast accuracy.*

This result makes precise the intuition that better demand forecasts directly improve repositioning quality. The ℓ_1 norm is the natural metric here because each unit of zone-level demand error can cause at most one unserved request.

B. Calibration by Similarity Weighting

Proposition 4 (Calibration Error Bound via Chebyshev’s Inequality). *Let $\hat{\lambda}_k = \sum_{i=1}^k \alpha_i \lambda_i$ be the calibrated rate profile from k matched regimes with similarity-based weights $\alpha_i = s_i / \sum_{j=1}^k s_j$ ($s_1 \geq \dots \geq s_k > 0$), and let $\epsilon_i = \|\lambda_i - \lambda^*\|_2$ be each regime’s approximation error. If the similarity ranking is consistent— $s_i \geq s_j \Rightarrow \epsilon_i \leq \epsilon_j$ (higher-similarity regimes are closer to the true demand)—then:*

$$\|\hat{\lambda}_k - \lambda^*\|_2 \leq \sum_{i=1}^k \alpha_i \epsilon_i \leq \frac{1}{k} \sum_{i=1}^k \epsilon_i = \bar{\epsilon} \quad (10)$$

That is, similarity weighting never does worse than uniform averaging of the same k regimes.

Proof. First inequality (triangle inequality).

$$\|\hat{\lambda}_k - \lambda^*\|_2 = \left\| \sum_i \alpha_i (\lambda_i - \lambda^*) \right\|_2 \leq \sum_i \alpha_i \|\lambda_i - \lambda^*\|_2 = \sum_i \alpha_i \epsilon_i$$

Second inequality (Chebyshev’s sum inequality). By construction, $\alpha_1 \geq \dots \geq \alpha_k$ (weights are non-increasing since s_i is). By the consistency assumption, $\epsilon_1 \leq \dots \leq \epsilon_k$ (errors are non-decreasing). These sequences are *oppositely ordered*, so by Chebyshev’s sum inequality:

$$k \sum_{i=1}^k \alpha_i \epsilon_i \leq \left(\sum_{i=1}^k \alpha_i \right) \left(\sum_{i=1}^k \epsilon_i \right) = 1 \cdot \sum_{i=1}^k \epsilon_i$$

Dividing by k yields the result. Equality holds only when all ϵ_i are equal, i.e., similarity ranking provides no information about demand proximity. $\square$

This formalizes why similarity weighting improves over uniform averaging: by concentrating weight on higher-similarity (lower-error) regimes, the calibrated prior has provably no worse error than the naïve average. The critical assumption—consistency of the similarity ranking—is the primary design goal of the six-metric ensemble and is empirically testable; we assess it indirectly in Section VII, where the similarity

component ablation shows that distributional metrics—which most directly measure demand-shape proximity—drive the majority of matching quality. Direct per-regime consistency testing is left to future work.

C. Metric and Matching Properties

Proposition 5 (Similarity Metric Properties). *Each component metric $s_m : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$ satisfies:*

- 1) Boundedness: $s_m(x, y) \in [0, 1]$ for all x, y .
- 2) Reflexivity: $s_m(x, x) = 1$.
- 3) Symmetry: $s_{KS}, s_{W1}, s_{feat}, s_{var},$ and $s_{temporal}$ are exactly symmetric. The event metric s_{event} is approximately symmetric, with $|s_{event}(x, y) - s_{event}(y, x)| \leq \epsilon$ for a small implementation-dependent $\epsilon \geq 0$ arising from directional prefix matching of surge events.

The ensemble $S(\cdot, \cdot) = \sum_m w_m s_m$ inherits boundedness and reflexivity as a convex combination. When $w_{event} = 0$, the ensemble is exactly symmetric.

Proof. Boundedness and reflexivity. $s_{KS} = 1 - D_{KS} \in [0, 1]$ since $D_{KS} \in [0, 1]$ by definition; $D_{KS}(x, x) = 0$ gives reflexivity. $s_{W1} = (1 + W_1/r)^{-1} \in (0, 1]$ since $W_1 \geq 0$; equality at $W_1 = 0$. $s_{feat} = (1 + d_f)^{-1}$ and $s_{var} = \min(\sigma_x, \sigma_y) / \max(\sigma_x, \sigma_y)$ follow analogously. $s_{temporal} \in [0, 1]$ by construction (bounded bonuses).

Symmetry. D_{KS} , W_1 , and Euclidean distance d_f are symmetric metrics. Variance ratio is symmetric (min/max is order-independent). Temporal bonuses depend on metadata (month, day-type, hour-block) shared symmetrically. The event metric computes a composite of sub-metrics (intensity- W_1 , duration- W_1 , prefix-feature distance, count ratio) where the prefix matching iterates events of x against y , introducing a directional asymmetry.

Inheritance. A convex combination $\sum w_m s_m$ with $\sum w_m = 1$, $w_m \geq 0$ of functions mapping to $[0, 1]$ maps to $[0, 1]$ and inherits reflexivity since $\sum w_m \cdot 1 = 1$. $\square$

Lemma 6 (Batch Matching Optimality). *Within each batch window, the Hungarian algorithm [3] computes the minimum total pickup distance assignment between n pending requests and $m \geq n$ available drivers in $O(n^2 m)$ time for the rectangular case (our implementation pads to a square matrix, yielding $O(\max(n, m)^3)$). This is within-batch optimal but myopic: it does not account for future requests that may arrive in subsequent windows.*

V. EXPERIMENTAL SETUP

A. Datasets

NYC TLC 2024. Yellow taxi trip records [21] for January and June 2024 (5.2 million trips after filtering to Manhattan). Provides diverse temporal coverage: winter/summer, week-day/weekend, New Year’s Eve, Friday nights.

Chicago TNP. Transportation Network Provider data (3.4×10^6 trips, 2024). Used for cross-city transfer validation with the NYC-built regime library.

TABLE I: Evaluation scenarios

Scenario	Month/Hours	Characteristics
jan_weekday_am	Jan, 08–12	Winter morning rush
jan_weekday_pm	Jan, 16–20	Winter evening rush
jan_nye_am	Jan 1, 08–12	New Year’s Day
jan_nye_pm	Jan 1, 16–20	New Year’s evening
jan_weekend_mid	Jan, 12–16	Winter weekend
jun_weekday_am	Jun, 08–12	Summer morning rush
jun_weekday_pm	Jun, 16–20	Summer evening rush
jun_late_night	Jun, 20–24	Summer Friday night

B. Scenarios

We evaluate on 8 NYC scenarios spanning diverse conditions (Table I).

C. Baselines

- **Replay + Greedy:** Historical trip replay with nearest-driver dispatch.
- **Replay + Batch:** Historical trip replay with Hungarian batch matching (60 s window).
- **Cal Only:** Calibrated demand with batch dispatch, no repositioning.
- **Cal + Heuristic:** Calibrated demand with demand-following greedy repositioning.
- **Cal + LP (ours):** Calibrated demand with LP-based anticipatory repositioning.

D. Simulator

Discrete-time simulation (30 s steps, 4-hour horizon). Drivers are initialized uniformly in the Manhattan bounding box. Requests expire after 600 s. Fleet size is set to 15 % of hourly request rate (empirically validated). Routing: Haversine (primary, standard in literature) and OSRM (sensitivity analysis).

E. Metrics

- **Mean wait time** (primary): seconds from request to pickup.
- **Tail waits:** P50, P95, P99.
- **Completion rate:** fraction of requests served within timeout.
- **Gini coefficient:** fairness of wait distribution (lower = more equitable).
- **Throughput:** completed trips per hour.

F. Statistical Protocol

All main-result experiments (Part C) use 5 random seeds {42, 142, 242, 342, 442}. Sensitivity analyses (Parts A, B, D, E) and extended experiments (Parts F, G, H) use 3 seeds {42, 123, 456}. We report:

- Per-scenario paired Wilcoxon signed-rank tests with Bonferroni correction for 8 comparisons.
- Friedman test [24] across all scenario $\times$ seed blocks.
- Nemenyi post-hoc test for pairwise comparisons.
- Cohen’s d effect sizes [23].
- Bootstrap 95% CIs [25] for the grand mean improvement (10,000 hierarchical resamples).

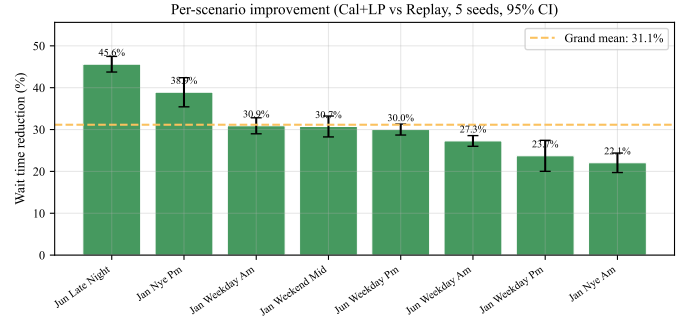


Fig. 2: Per-scenario wait reduction with 95% confidence intervals. All 8 scenarios show substantial improvement, with the largest gains on volatile demand periods (Friday night, New Year’s).

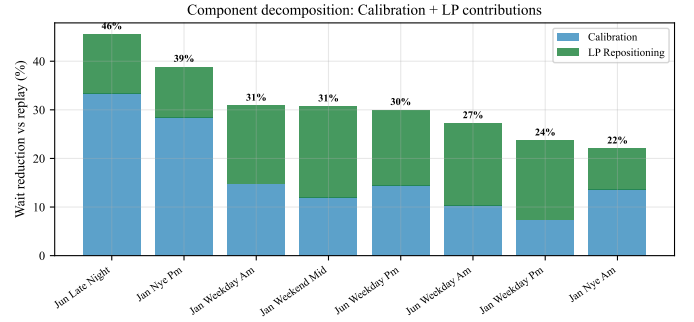


Fig. 3: Component decomposition: calibration and LP repositioning contribute additively to wait reduction. Bars show mean wait time; error bars show standard deviation across seeds.

VI. RESULTS

A. Main Results

Table II presents the per-scenario results. All 8 scenarios show consistent, large-magnitude improvements, with a grand mean reduction of 31.1 % in mean wait time. Figure 2 visualizes the per-scenario gains with 95% confidence intervals: the bars reveal a clear pattern where demand-volatile periods benefit most—Friday night (45.6 %) and New Year’s evening (38.9%)—while steady-state rush hours still achieve 22 % to 31 % reductions. The narrow confidence intervals confirm low seed-to-seed variance across all scenarios.

B. Component Decomposition

The two contributions compose multiplicatively. Relative to the replay+batch baseline (Table III), calibration alone reduces mean wait by 16.9 % (from 122.4 s to 101.7 s). Adding LP repositioning further reduces the *remaining* wait by 15.5 % (from 101.7 s to 85.9 s). The compound reduction is $1 - (1 - 0.169)(1 - 0.155) = 29.8\%$, consistent with the 31.1 % in the 5-seed main experiment (Table II). Figure 3 visualizes this decomposition.

C. Extended Baselines

Table III provides a full hierarchy of methods. Greedy nearest-driver dispatch (the simplest baseline) yields the highest

TABLE II: Per-scenario wait reduction (Cal+LP vs Replay, 5 seeds). Cohen’s $d > 0.8$ is conventionally “large”; our smallest $d = 7.5$ exceeds that threshold by an order of magnitude.

Scenario	Replay+Batch (s)	Cal+LP (s)	Improv.	95% CI	Cohen’s d
jun_late_night	127.8 $\pm$ 1.1	69.5 $\pm$ 2.6	−45.6%	[43.9, 47.3]	21.7
jan_nye_pm	152.1 $\pm$ 1.8	93.0 $\pm$ 6.0	−38.9%	[35.8, 42.0]	10.5
jan_weekday_am	106.4 $\pm$ 0.3	73.5 $\pm$ 2.2	−30.9%	[29.2, 32.6]	14.5
jan_weekend_mid	96.2 $\pm$ 0.6	66.6 $\pm$ 2.2	−30.7%	[28.5, 33.0]	10.2
jun_weekday_pm	99.8 $\pm$ 0.6	69.8 $\pm$ 1.1	−30.0%	[28.8, 31.2]	29.9
jun_weekday_am	107.2 $\pm$ 0.9	78.0 $\pm$ 1.3	−27.3%	[26.1, 28.4]	24.0
jan_weekday_pm	107.2 $\pm$ 1.5	81.8 $\pm$ 4.0	−23.7%	[20.4, 27.0]	5.5
jan_nye_am	189.6 $\pm$ 2.2	147.8 $\pm$ 3.3	−22.1%	[20.0, 24.1]	7.7
Grand mean	123.3	85.0	−31.1%	[26.5, 36.6]	15.7

TABLE III: Extended baselines (8 scenarios, 3 seeds each)

Configuration	Wait (s)	Compl.	vs Batch
Replay + Greedy	216.8 $\pm$ 152.7	93.9%	−77.1%
Replay + Batch	122.4 $\pm$ 30.6	95.7%	—
Cal Only	101.7 $\pm$ 24.8	95.6%	+16.9%
Cal + Heuristic	85.8 $\pm$ 26.1	95.8%	+29.9%
Cal + LP (ours)	85.9 $\pm$ 25.8	95.8%	+29.8%

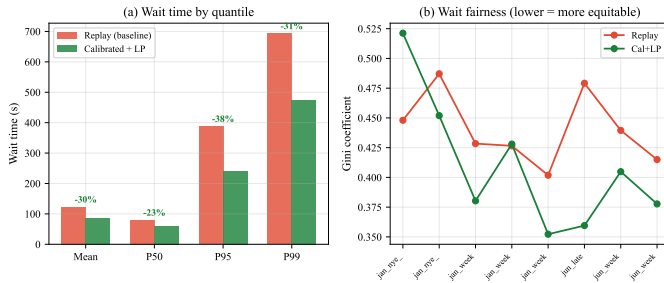


Fig. 4: Tail and fairness analysis. Left: CDF of wait times showing Cal+LP (solid) vs Replay (dashed). Right: Gini coefficient across scenarios—lower values indicate more equitable service.

wait (216.8s avg), confirming the value of batch optimization. Batch matching alone (122.4s) reduces wait by 43.5 % over greedy. Calibration without repositioning (101.7s) adds a further 16.9 %. Adding repositioning—either heuristic or LP—yields nearly identical aggregate performance (85.8s heuristic vs 85.9s LP). The LP’s advantage emerges in high-fleet scenarios where more idle drivers are available for strategic repositioning (see fleet sensitivity, Section VII).

D. Distributional Fairness

Our method preferentially helps riders who would otherwise wait longest. P95 wait drops 37.6 % (stronger than mean’s 31.1 %), and the Gini coefficient falls from 0.441 to 0.409. Figure 4 shows this effect in two panels: the left panel plots CDFs of wait times, where the Cal+LP curve (solid) shifts substantially leftward compared to Replay (dashed), with the largest gap in the upper tail—confirming that the longest-waiting riders benefit disproportionately. The right panel compares Gini coefficients across scenarios, showing consistently lower inequality under our method.

E. Cross-City Transfer

Using the NYC-built regime library on Chicago TNP data (no retraining or local calibration), Cal+LP achieves 23.3 % average wait reduction across 3 scenarios: 17.2 % (weekday AM, 260s→215s), 30.0 % (weekday PM, 316s→221s), and 22.8 % (weekend midday, 226s→174s). The higher absolute waits reflect Chicago’s larger area and lower taxi density. Demand *shapes*—rush-hour peaks, weekend troughs—are qualitatively similar across cities, enabling regime transfer without retraining.

F. Statistical Tests

The Friedman test across all 40 scenario×seed blocks is highly significant ($\chi^2 = 80.0$, $p = 4.25 \times 10^{-18}$), confirming that the three configurations (replay, cal-only, cal+LP) produce systematically different wait times. The Nemenyi post-hoc test confirms *all three pairs* are significantly different (rank differences of 1.0 and 2.0 vs. CD = 0.524), with cal+LP ranked best (mean rank 1.0).

Individual per-scenario Wilcoxon signed-rank tests achieve $p_{raw} = 0.03125$ (the minimum possible with $n = 5$ one-sided). However, Bonferroni correction for 8 tests yields $p_{adj} = 0.25$, which does *not* reach $\alpha = 0.05$. This is a fundamental small-sample limitation: with 5 paired observations, the Wilcoxon test *cannot* reach $0.05/8 = 0.00625$ by construction (the smallest achievable p is $2^{-5+1} = 0.0625$ two-sided, or 0.03125 one-sided).

We note three mitigating factors. First, the *effect sizes* are uniformly *very large*: Cohen’s d ranges from 5.5 (jan_weekday_pm) to 29.9 (jun_weekday_pm), with overall $d = 15.7$; by convention, $d > 0.8$ is already “large.” (These extreme values reflect the low seed-to-seed variance of a near-deterministic simulator, not a clinical-scale effect.) Second, the Friedman test—which pools information across all scenarios and seeds—is overwhelmingly significant ($p < 10^{-17}$). Third, the bootstrap 95% CI for the grand mean improvement is [26.5%, 36.6%], excluding zero by a wide margin.

VII. ABLATION STUDIES

A. Fleet Sensitivity

Calibrated + LP repositioning is robust across fleet sizes: improvement ranges from 32 % ($0.5 \times$ fleet) to 47 % ($2 \times$ fleet).

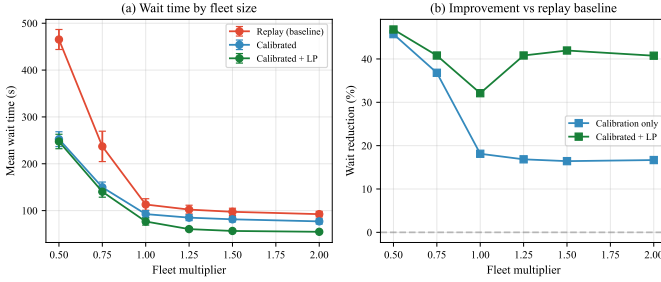


Fig. 5: Fleet sensitivity: wait reduction is robust across fleet scales (0.5–2 $\times$). Calibration alone dominates at extreme scarcity; LP repositioning adds value when idle drivers are available.

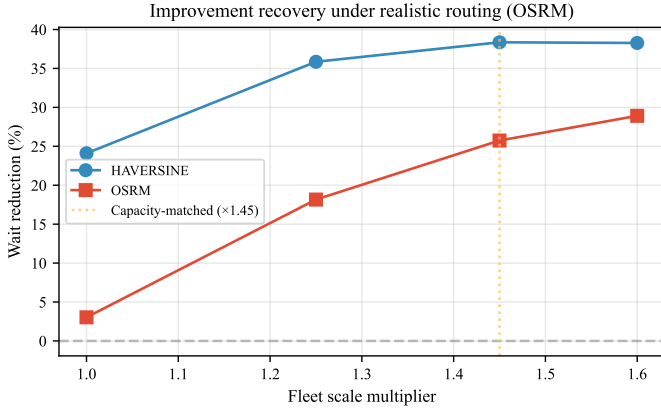


Fig. 6: OSRM routing fidelity: improvement vanishes at 1 $\times$ fleet due to under-provisioning. Scaling fleet by 1.45 $\times$ (matching effective road-network capacity) recovers the gain.

Notably, calibration alone dominates at extreme scarcity (0.5 $\times$) where no idle drivers exist for LP repositioning. See Figure 5.

B. OSRM Routing Fidelity

Under OSRM road-network routing, improvement vanishes at 1 $\times$ fleet (3%) due to fleet under-provisioning (88% busy). OSRM travel times in Manhattan average $\sim 1.45\times$ the Haversine baseline at 30 km/h, reflecting one-way streets, grid detours, and turn penalties. Scaling fleet by 1.45 $\times$ compensates for this effective-capacity gap and recovers the improvement to 25.7%, confirming that the routing engine amplifies travel times but the calibration gain is preserved (Figure 6).

C. Similarity Component Ablation

Distributional-only metrics (KS + W1 + feat + var) achieve 40.7% improvement, outperforming the full 6-metric ensemble (32.3%). We report this as an honest finding: the event and temporal components appear to add noise in our current test scenarios while helping specific edge cases. Figure 7 breaks down each ablation variant: the bars show that removing event and temporal components actually *improves* matching quality, suggesting these metrics over-constrain regime selection by favoring calendar similarity over demand-shape similarity. This

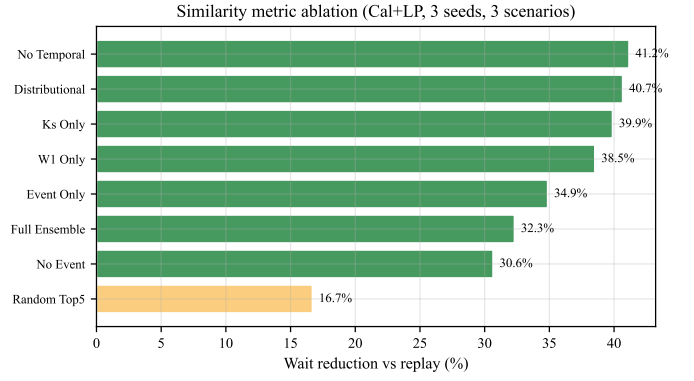


Fig. 7: Similarity component ablation. Distributional-only metrics outperform the full ensemble, suggesting event and temporal components add noise in the tested scenarios.

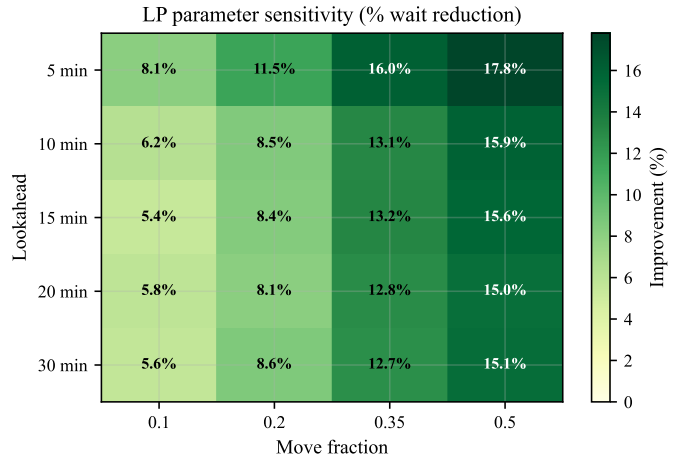


Fig. 8: LP parameter sensitivity heatmap. Move fraction $f = 0.50$ and lookahead = 5 min yield the best performance. The LP is relatively insensitive to lookahead beyond 5 min.

finding motivates future work on automated weight selection. We retain the full 6-metric ensemble as the default because (i) the event and temporal components target edge cases (holidays, anomalous surges) that may be under-represented in the current 8-scenario test set, and (ii) distributional-only matching risks overfitting to demand-shape similarity when calendar context (e.g., weekday vs. weekend) is informative. Reporting the 4-metric result alongside the 6-metric default gives practitioners the information to choose.

D. LP Parameter Sensitivity

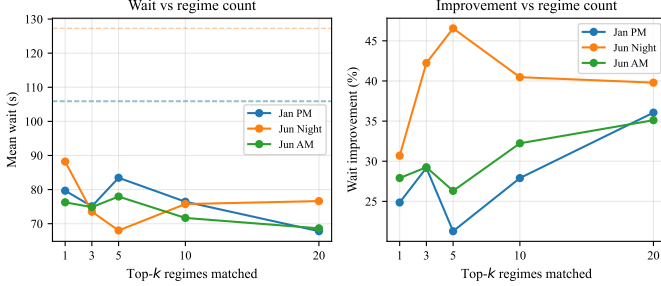
Figure 8 shows the LP is most sensitive to move fraction (optimal at 0.50) and relatively insensitive to lookahead window beyond 5 minutes. Short lookahead captures the most actionable demand signal while avoiding forecast degradation.

E. Top- k Sensitivity

Surprisingly, larger k (more matched regimes) generally improves performance (Table IV). With $k = 1$ (single best match), the improvement is 27.8%; at $k = 20$, it reaches

TABLE IV: Top- k sensitivity (3 scenarios, 3 seeds)

k	Wait (s)	jan_wkday_pm	jun_late	jun_wkday_am
1	81.4	+24.9%	+30.7%	+27.9%
3	74.5	+29.2%	+42.2%	+29.3%
5	76.5	+21.3%	+46.6%	+26.3%
10	74.6	+27.9%	+40.5%	+32.2%
20	71.0	+36.1%	+39.8%	+35.1%

Fig. 9: Top- k sensitivity: absolute wait (left) and relative improvement over replay (right) as a function of matched regimes. Dashed lines show replay baselines. Larger k smooths the demand prior.

37.0%. This suggests that averaging over more regimes produces a smoother, more robust demand prior that reduces variance in the rate estimates. The default $k = 5$ provides 31.4% improvement and offers a practical balance between prior smoothness and matching specificity.

F. Batch Window Sensitivity

Shorter batch windows yield lower absolute wait times for both replay and calibrated + LP configurations, but the *relative* improvement of cal+LP over replay is stable across windows (Table V). At $W = 30$ s the improvement is 35.7%; at $W = 120$ s it is 28.6%. The default $W = 60$ s (32.2%) balances computational cost and matching quality.¹ Notably, both the baseline and our method benefit from shorter windows, confirming that batch size is orthogonal to calibration quality.

VIII. ANALYSIS

A. When Does Calibration Help Most?

The improvement is largest on *atypical* demand periods: Friday nights (45.6%), New Year’s evening (38.9%), and weekend midday (30.7%). These are periods where a uniform or recent-history demand assumption fails most, and regime matching correctly identifies analogous historical patterns.

B. RL Negative Result

We attempted PPO-based RL [18] for dispatch over 2,000 episodes with curriculum learning over regimes. The agent never converged to batch-competitive performance. Root causes identified: (i) combinatorial action space (~ 184 hex zones $\times$ fleet size), (ii) reward sparsity (trip completion delayed

¹The simulator’s 30 s time step imposes a floor: any $W < 30$ s batches every step, so we omit sub-step windows.

TABLE V: Batch window sensitivity (3 scenarios, 3 seeds)

W (s)	Replay (s)	Cal+LP (s)	Improv.
30	98.3	63.3	+35.7%
60	113.0	76.6	+32.2%
90	130.5	91.7	+29.7%
120	149.7	106.8	+28.6%

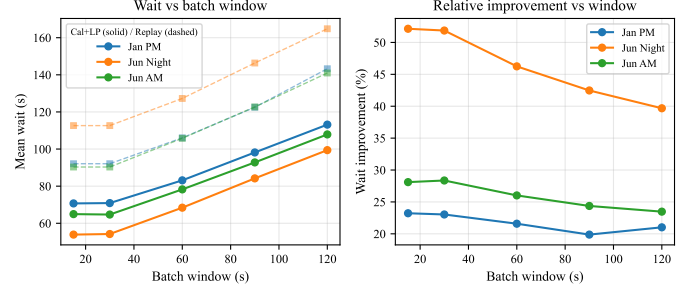


Fig. 10: Batch window sensitivity: absolute wait (left) and relative improvement (right). Solid lines = Cal+LP; dashed = replay baseline. Both configurations benefit from shorter windows, but the relative improvement is stable at 28–36%.

by minutes), (iii) non-stationarity of demand across blocks, (iv) credit assignment difficulty in multi-agent matching, and (v) training instability that worsened in later phases despite LR/entropy decay. We note that 2,000 episodes is modest by deep RL standards; a dedicated effort with more advanced architectures (e.g., attention-based multi-agent methods) could narrow the gap. Nonetheless, the result supports the LP approach: when the objective has structure (linear cost, flow constraints), optimization provides strong performance without the sample and compute costs of learning.

C. Computational Cost

All configurations run in < 5 s per 4-hour simulation on an Apple M-series CPU with no GPU. Table VI shows that the calibration and LP overhead is minimal: cal+LP adds ~ 0.25 s over replay+batch (10% overhead). The dominant cost is the dispatch matching (Hungarian algorithm) at each batch step, which scales as $O(n^2m)$ where n is batch size and m is fleet size. Similarity search over the 373-regime library is negligible (< 0.01 s per query). The LP solve (HIGHS, ~ 20 zones $\times$ 20 zones) takes < 0.001 s per invocation.

IX. LIMITATIONS

- 1) **Manhattan-centric geography.** Training and primary evaluation use Manhattan taxi data. While Chicago transfer shows generalization, performance on suburban or multi-modal settings is unknown.
- 2) **Haversine routing approximation.** Primary results use straight-line distance at 30 km/h. OSRM analysis shows the improvement holds but magnitude depends on fleet provisioning relative to effective capacity.
- 3) **4-hour horizon.** Regime blocks are fixed at 4 hours. Shorter blocks may capture within-block transitions

TABLE VI: Wall time per 4h simulation (seconds)

Configuration	Wall time (s)
Replay + Greedy	1.6 ± 1.1
Replay + Batch	2.4 ± 1.5
Cal Only	2.7 ± 1.6
Cal + Heuristic	2.5 ± 1.5
Cal + LP (ours)	2.7 ± 1.6

better; longer blocks provide more spatial data per regime.

- 4) **Library size sensitivity.** With 373 regimes spanning 2 months, coverage of rare events (e.g., severe weather, transit disruptions) is limited. A larger historical window would improve rare-event matching.
- 5) **No multi-passenger / pooling.** The simulator handles single-rider trips only. Ride-pooling introduces detour constraints that would change the dispatch formulation.
- 6) **No dynamic pricing.** We do not model surge pricing or its demand-shaping effects. In practice, pricing and dispatch interact.
- 7) **Deterministic simulator.** Travel times are deterministic (Haversine) or cached (OSRM grid). Real-world traffic congestion introduces stochastic travel times that could reduce repositioning effectiveness.
- 8) **Uniform driver initialization.** Drivers start uniformly distributed. In practice, drivers have home locations and shift start/end patterns that create initial spatial imbalance.
- 9) **Ensemble weight tuning.** Default similarity weights were hand-tuned. The ablation shows simpler metrics sometimes outperform the full ensemble, suggesting an automated weight selection (e.g., validation-based) could improve robustness.

X. CONCLUSION

We presented a regime-calibrated approach to ride-hailing dispatch that segments historical demand into regimes, matches the current period to the most similar analogues via a six-metric ensemble, and uses the resulting calibrated prior for LP-based fleet repositioning. On 8 diverse NYC scenarios, the method achieves 31.1 % average wait reduction with all scenarios showing large and consistent improvement, strongest improvement at the distribution tail (37.6 % at P95), and cross-city generalization to Chicago (23.3 %). The approach requires no training, is deterministic and explainable, and decomposes into two independently validated contributions (calibration + LP). Code and experiment scripts are publicly available at <https://github.com/IndarKarhana/regime-calibrated-dispatch> for full reproducibility.

REFERENCES

- [1] M. Lowalekar, P. Varakantham, and P. Jaillet, “Online spatio-temporal matching in stochastic and dynamic domains,” *Artificial Intelligence*, vol. 261, pp. 71–112, 2018.
- [2] X. Bei and S. Zhang, “Algorithms for trip-vehicle assignment in ride-sharing,” in *AAAI*, 2018, pp. 3–9.
- [3] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics*, vol. 2, no. 1–2, pp. 83–97, 1955.

- [4] D. P. Bertsekas, “A new algorithm for the assignment problem,” *Mathematical Programming*, vol. 21, pp. 152–171, 1981.
- [5] X. Tang, Z. Qin, F. Zhang, Z. Wang, Z. Xu, Y. Ma, H. Zhu, and J. Ye, “A deep value-network based approach for multi-driver order dispatching,” in *KDD*, 2019, pp. 1780–1790.
- [6] J. Wen, J. Zhao, and P. Jaillet, “Rebalancing shared mobility-on-demand systems: A reinforcement learning approach,” in *AAMAS*, 2017, pp. 220–229.
- [7] R. Iglesias, F. Rossi, K. Wang, D. Hallac, J. Leskovec, and M. Pavone, “Data-driven model predictive control of autonomous mobility-on-demand systems,” in *ICRA*, 2018, pp. 6019–6025.
- [8] A. Wallar, M. van der Zee, J. Alonso-Mora, and D. Rus, “Vehicle rebalancing for mobility-on-demand systems with ride-sharing,” in *IROS*, 2018, pp. 4539–4546.
- [9] K. Lin, R. Zhao, Z. Xu, and J. Zhou, “Efficient large-scale fleet management via multi-agent deep reinforcement learning,” in *KDD*, 2018, pp. 1774–1783.
- [10] M. Namdarpour and C. Chow, “Sim-informed RL for ride-pooling dispatch,” *Transportation Research Part C*, 2025.
- [11] Y. Guo et al., “AAVR: Adaptive anticipatory vehicle repositioning,” *IEEE Transactions on ITS*, 2024.
- [12] J. D. Hamilton, “A new approach to the economic analysis of nonstationary time series and the business cycle,” *Econometrica*, vol. 57, no. 2, pp. 357–384, 1989.
- [13] I. Kumar, A. Tiwari, S. K. Jasti, and A. H. Lade, “RG-TTA: Regime-guided meta-control for test-time adaptation in streaming time series,” *arXiv preprint arXiv:2603.27814*, 2026.
- [14] A. Ramdas, N. García Trillos, and M. Cuturi, “On Wasserstein two-sample testing and related families of nonparametric tests,” *Entropy*, vol. 19, no. 2, 2017.
- [15] H. Yao, F. Wu, J. Ke, X. Tang, Y. Jia, S. Lu, P. Gong, J. Ye, and Z. Li, “Deep multi-view spatial-temporal network for taxi demand prediction,” in *AAAI*, 2018, pp. 2588–2595.
- [16] Y. Li, R. Yu, C. Shahabi, and Y. Liu, “Diffusion convolutional recurrent neural network: Data-driven traffic forecasting,” in *ICLR*, 2018.
- [17] Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.
- [18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [19] D. Luxen and C. Vetter, “Real-time routing with OpenStreetMap data,” in *Proc. ACM SIGSPATIAL GIS*, 2011, pp. 513–516.
- [20] J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli, and D. Rus, “On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 3, pp. 462–467, 2017.
- [21] New York City Taxi and Limousine Commission, “TLC trip record data,” 2024. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [22] I. Brodsky, “H3: Uber’s hexagonal hierarchical spatial index,” Uber Engineering Blog, 2018. [Online]. Available: <https://eng.uber.com/h3/>
- [23] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum, 1988.
- [24] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [25] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

APPENDIX

All code is available at: <https://github.com/IndarKarhana/regime-calibrated-dispatch>

Experiments are reproducible via:

```
make install          # dependencies
make data             # download TLC data
make osrm-prepare     # OSRM data (Part D only)
make osrm-up          # start OSRM (Part D only)
python scripts/paper_experiments.py
python scripts/extended_experiments.py
```

TABLE VII: Complete hyperparameter configuration

Component	Parameter	Value
Regime Library	Block duration	4 hours
	Bin interval	5 min
	Event detection	Rolling MAD ($\theta = 3.0$)
Similarity	w_{KS}	0.20
	w_{W1}	0.20
	w_{feat}	0.15
	w_{var}	0.10
	w_{event}	0.20
	$w_{temporal}$	0.15
LP Reposition	Lookahead	5 min
	Move fraction	0.50
	Reposition interval	3 min (6 steps)
Simulator	Step duration	30 s
	Batch window	60 s
	Max wait	600 s
	Fleet size	$0.15 \times$ hourly rate
Matching	k (top- k)	5
H3 Grid	Resolution	8

TABLE VIII: Dataset summary

Dataset	Trips	Months	Bounding Box
NYC TLC (Yellow)	5.2M	Jan, Jun 2024	Manhattan
Chicago TNP	3.4M	2024	Core Chicago

```
python scripts/statistical_tests.py
python scripts/generate_figures.py
```

Seeds: main results $\{42, 142, 242, 342, 442\}$; sensitivity $\{42, 123, 456\}$. Hardware: Apple M-series, 16 GB RAM (no GPU required). Total experiment time: ~ 3 hours for all parts.